

Лабораторная работа №2

Задача о погоне

Нефедова Н.Н.

6 марта 2024

Российский университет дружбы народов, Москва, Россия

Информация

- Нефедова Наталия Николавна
- Студент
- Обучающийся на кафедре математического моделирования и искусственного интеллекта
- Российский университет дружбы народов
- <https://github.com/nnnefedova>

Вводная часть

Решить задачу о погоне и изучить основы языка программирования Julia.

1. Запишите уравнение, описывающее движение катера, с начальными условиями для двух случаев (в зависимости от расположения катера относительно лодки в начальный момент времени).
2. Постройте траекторию движения катера и лодки для двух случаев.
3. Найдите точку пересечения траектории катера и лодки.

Выполнение лабораторной работы

Выполнение лабораторной работы

Рассчитаем свой вариант по формуле и получаем наш вариант - 58

```
main.py
1 def определить_номер_задания(номер_студбилета, количество_заданий):
2     # Вычисляем номер задания по формуле
3     номер_задания = (номер_студбилета % количество_заданий) + 1
4     return номер_задания
5
6 # Пример использования функции
7 номер_студбилета = int(input("Введите номер студенческого билета: "))
8 количество_заданий = int(input("Введите количество заданий: "))
9
10 номер_задания = определить_номер_задания(номер_студбилета, количество_заданий)
11 print(f"Номер задания: {номер_задания}")
12
```


1. На море в тумане катер береговой охраны преследует лодку браконьеров.

Через определенный промежуток времени туман рассеивается, и лодка обнаруживается на расстоянии 20,2 км от катера. Затем лодка снова скрывается в тумане и уходит прямолинейно в неизвестном направлении. Известно, что скорость катера в 5,1 раза больше скорости браконьерской лодки.

2. Примем за момент отсчета времени момент первого рассеивания тумана. Введем полярные координаты с центром в точке нахождения браконьеров и осью, проходящей через катер береговой охраны. Тогда начальные координаты катера $(20, 2; 0)$. Обозначим скорость лодки v .

3. Траектория катера должна быть такой, чтобы и катер, и лодка все время были на одном расстоянии от полюса. Только в этом случае траектория катера пересечется с траекторией лодки. Поэтому для начала катер береговой охраны должен двигаться некоторое время прямолинейно, пока не окажется на том же расстоянии от полюса, что и лодка браконьеров. После этого катер береговой охраны должен двигаться вокруг полюса удаляясь от него с той же скоростью, что и лодка браконьеров.

4. Пусть время t - время, через которое катер и лодка окажутся на одном расстоянии от начальной точки.

$$t = \frac{x}{v}$$

$$t = \frac{20.2 - x}{5.1v}$$

$$t = \frac{20.2 + x}{5.1v}$$

Из этих уравнений получаем объединение двух уравнений:

$$\left\{ \begin{array}{l} \frac{x}{v} = \frac{20.2 - x}{5.1v} \\ \frac{x}{v} = \frac{20.2 + x}{5.1v} \end{array} \right.$$

Решая это, получаем два значения для x :

$$x_1 = 3.311$$

$$x_2 = 4.927$$

$$v_\tau$$

– тангенциальная скорость

$$v$$

– радиальная скорость

$$v = \frac{dr}{dt}$$

$$v_\tau = \frac{\sqrt{(5.1v)^2 - v^2}}{10} = \frac{\sqrt{2501}v}{10}$$

Уравнение

$$\left\{ \begin{array}{l} \frac{dr}{dt} = v \\ r \frac{d\theta}{dt} = \frac{\sqrt{2501}v}{10} \end{array} \right.$$

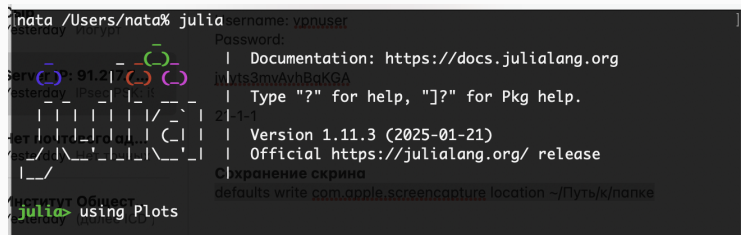
Итоговое уравнение после того, как убрали производную по t :

$$\frac{dr}{d\theta} = \frac{10r}{\sqrt{2501}}$$

Работа с Julia

1. Скачиваем Julia .

2. Запускаем Julia .



```
julia> using Plots
```

Рис. 2: Запуск Julia

3. Процесс запуска Julia

```
nata /Users/nata% julia

      _       _      _ _      _        _
     /_   _/   _/   _/_/   _/_/   _/
    /__/_/___/_/___/_/___/_/___/_/___/
   /__/_/___/_/___/_/___/_/___/_/___/
  /__/_/___/_/___/_/___/_/___/_/___/
 /__/_/___/_/___/_/___/_/___/_/___/
/_/___/_/___/_/___/_/___/_/___/_/___/

Documentation: https://docs.julialang.org
Type "?" for help, "]"? for Pkg help.
Version 1.11.3 (2025-01-21)
Official https://julialang.org/ release

savefig(plt, "lab2_01.png")
problem = ODEProblem(F, r0_2, T_2) result = so
plot1 = plot(prof=polar, aspect_ratio=:equal, dpi =
plot!(plot1, xlabel="theta", ylabel="r(t)", title="Случ
```

Рис. 3: Процесс запуска

4. Скачаем необходимые для работы пакеты

```
[4607b0f0] + SuiteSparse
Info Packages marked with x have new versions available but compatibility constraints restrict them from upgrading. To see why use `status --outdated -m`
Precompiling DifferentialEquations...
Progress [=====>] 206/247
  ○ Sundials
    ○ OrdinaryDiffEqTsit5
    ○ OrdinaryDiffEqVerner
    ✓ OrdinaryDiffEqRKN
    ✓ OrdinaryDiffEqFeagin
    ○ OrdinaryDiffEqQPRK
    ○ OrdinaryDiffEqLowStorageRK
```

Рис. 4: Скачивание необходимых для работы пакетов

5. Запуск кода

```
julia> rAngles = [result.t[dxR] for i in 1:size(result.t)[1]]
11-element Vector{Float64}:
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603
-0.7979859624035603

julia> plt1 = plot(proj=:polar, aspect_ratio=:equal, dpi = 1000, legend=true, bg=:white)

julia> plot!(plt1, xlabel="theta", ylabel="r(t)", title="Случай номер 2", legend=:outerbottom)

julia> plot!(plt1, [rAngles[1], rAngles[2]], [0.0, result.u[size(result.u)[1]]], label="Путь
лодки", color=:blue, lw=1)

julia> scatter!(plt1, rAngles, result.u, label="", mc=:blue, ms=0.0005)

julia> plot!(plt1, result.t, result.u, xlabel="theta", ylabel="r(t)", label="Путь катера", co
lor=:green, lw=1)

julia> scatter!(plt1, result.t, result.u, label="", mc=:green, ms=0.0005)

julia> savefig(plt1, "lab2_02.png")
```

Рис. 5: Запуск кода

6. Просмотр результата работы.

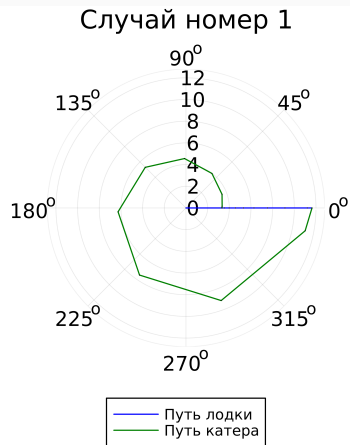


Рис. 6: Случай 1

7. Просмотр результата работы.

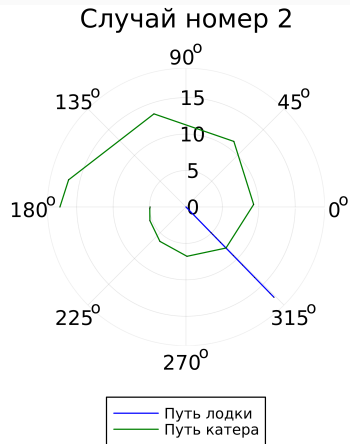


Рис. 7: Случай 2

Были изучены основы языка программирования Julia и его библиотеки, которые используются для построения графиков и решения дифференциальных уравнений. А также решили задачу о погоне.

[1] Документация по Julia: <https://docs.julialang.org/en/v1/>

[2] Решение дифференциальных уравнений: <https://www.wolframalpha.com/>